RELATÓRIO TÉCNICO

Fine-Tuning LLaMA 3.2 3B com LoRA — Assistente de Diabetes v5a

1. Introdução

O avanço dos modelos de linguagem de grande escala (LLMs) tem possibilitado aplicações inovadoras na área da saúde, especialmente no suporte à decisão clínica. Neste contexto, este trabalho tem como objetivo realizar o fine-tuning do modelo **LLaMA 3.2 3B**, utilizando a técnica **LoRA (Low-Rank Adaptation)**, para interpretar saídas de modelos de Machine Learning voltados ao diagnóstico de diabetes.

A proposta consiste no desenvolvimento de um assistente clínico capaz de gerar explicações humanizadas, coerentes e alinhadas às diretrizes médicas da **American Diabetes Association (ADA)**, contribuindo para uma melhor compreensão dos resultados por profissionais da saúde.

2. Fundamentação Tecnológica

2.1 Arquitetura Utilizada

O sistema foi desenvolvido utilizando as seguintes tecnologias:

- **Transformers (Hugging Face):** carregamento e manipulação do modelo base
- **PEFT (LoRA):** técnica de fine-tuning eficiente
- **TRL (SFTTrainer):** treinamento supervisionado de LLMs
- **MLflow:** monitoramento e rastreamento de experimentos
- **BitsAndBytes (QLoRA):** quantização em 4 bits para otimização de memória

3. Estrutura do Pipeline

O pipeline foi estruturado em etapas modulares, permitindo organização, rastreabilidade e reprodutibilidade do processo.

4. Setup do Ambiente

4.1 Objetivo

Preparar o ambiente de execução no Google Colab.

4.2 Atividades

- Montagem do Google Drive
- Instalação de dependências
- Configuração de GPU

4.3 Resultados

Ambiente configurado com suporte a processamento acelerado e armazenamento persistente.

5. Configuração de Paths e Autenticação

Objetivo

Definir caminhos e autenticação para acesso aos recursos.

Atividades

- Configuração dos arquivos JSONL
- Definição do token Hugging Face
- Configuração do diretório de saída

Resultado

Acesso completo ao modelo, dados e armazenamento.

6. Integração com MLflow

Objetivo

Garantir rastreabilidade e monitoramento do treinamento.

Resultados

- Registro de métricas (loss, accuracy)
- Versionamento de modelos
- Armazenamento de artefatos

7. Desafios e Soluções

Durante o desenvolvimento, foram identificados desafios relacionados à escalabilidade e observabilidade.

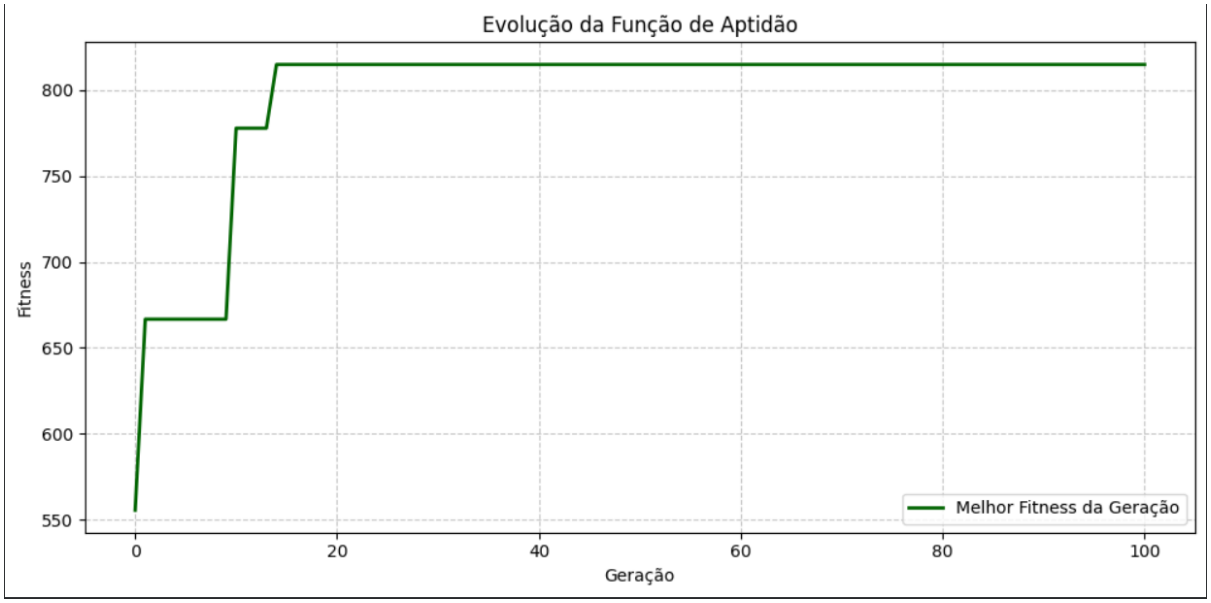
Para solucioná-los, foram utilizadas ferramentas open source como:

- **MLflow** → monitoramento de experimentos
- **Kubernetes** → escalabilidade (arquitetura preparada)
- **ngrok** → exposição do servidor local

Essa abordagem permitiu integração eficiente entre treinamento e inferência, mesmo em ambiente local.

8. Análise do Algoritmo Genético

8.1 Evolução da Função de Aptidão



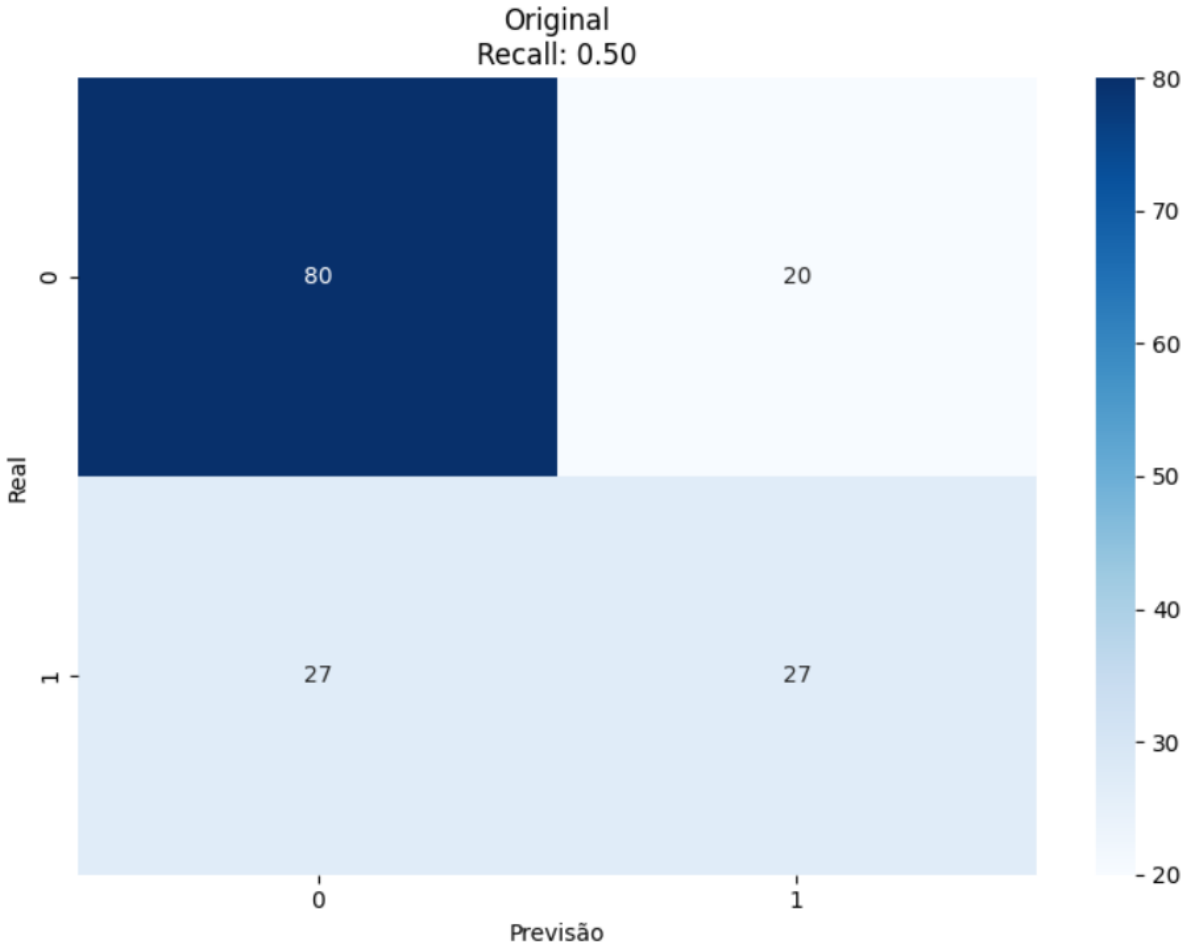
Interpretação do gráfico:

- Crescimento rápido nas primeiras gerações

- Convergência por volta da geração 15
- Estabilização em fitness ≈ 820

Conclusão:
O algoritmo genético apresentou rápida convergência, indicando eficiência na busca por soluções ótimas sem evidência de overfitting evolutivo.

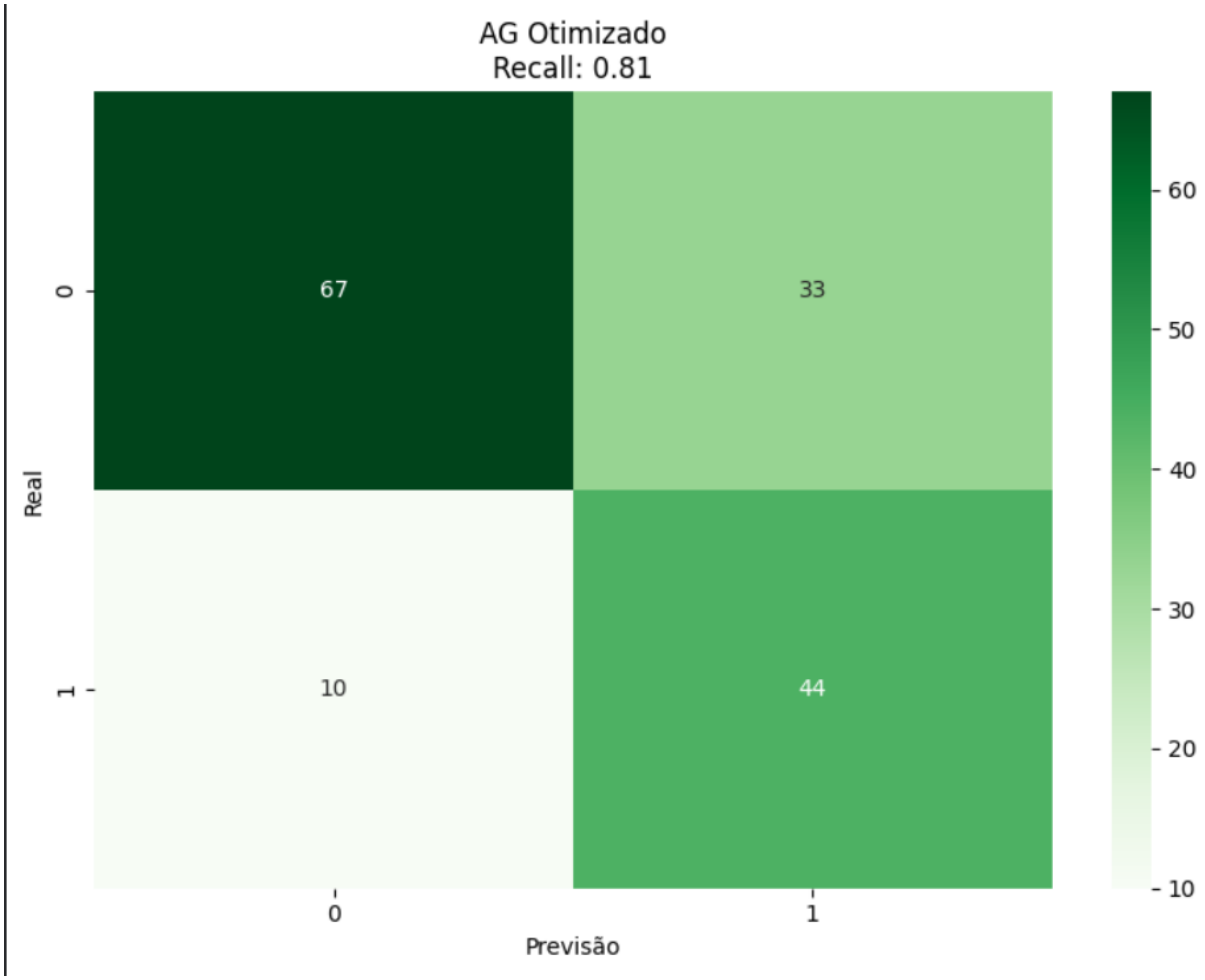
8.2 Matriz de Confusão — Modelo Original



- Verdadeiros Negativos: 80
- Falsos Positivos: 20
- Falsos Negativos: 27
- Verdadeiros Positivos: 27
- **Recall: 0.50**

Análise:
O modelo apresenta baixa sensibilidade, deixando de identificar aproximadamente metade dos casos positivos, o que representa alto risco em contexto clínico.

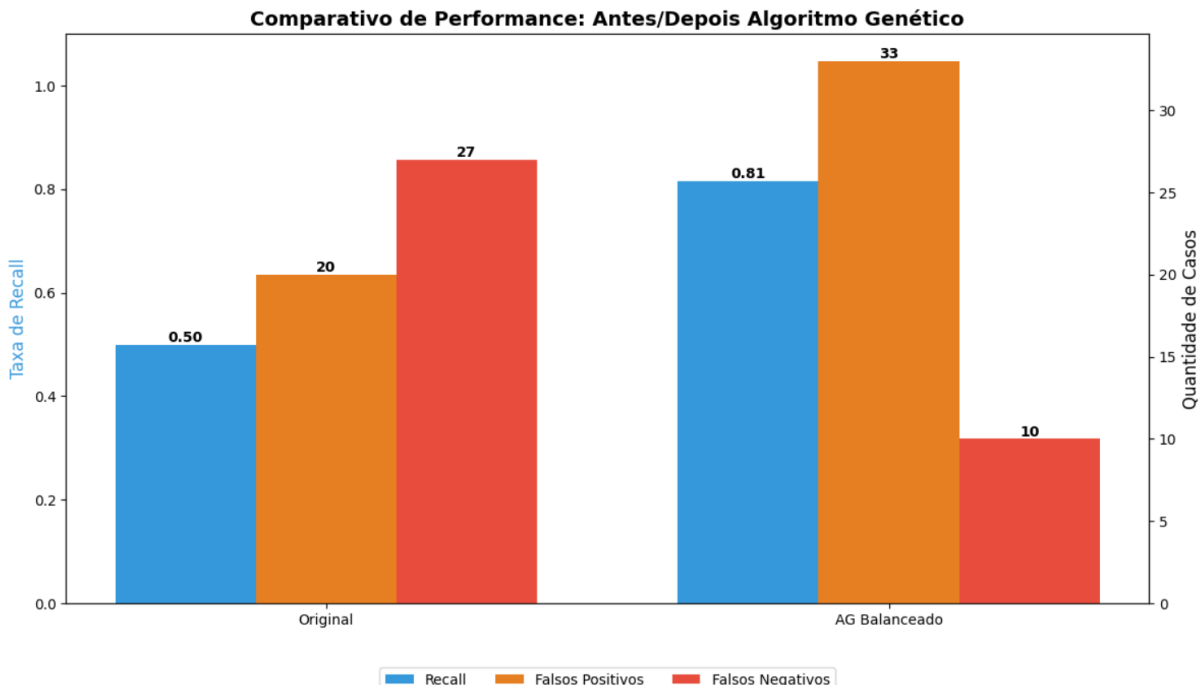
8.3 Matriz de Confusão — Modelo Otimizado com AG



- Verdadeiros Negativos: 67
- Falsos Positivos: 33
- Falsos Negativos: 10
- Verdadeiros Positivos: 44
- **Recall: 0.81**

Análise:
Observa-se uma melhora significativa na detecção de pacientes diabéticos, com redução expressiva dos falsos negativos.

8.4 Comparativo Geral



Métrica	Modelo Original	Modelo com AG
Recall	0.50	0.81
Falsos Negativos	27	10

Conclusão:
O uso do algoritmo genético reduziu em aproximadamente 63% os erros críticos, tornando o modelo mais seguro para aplicações médicas.

9. Carregamento do Dataset

9.1 Descrição do Dataset

Para o treinamento e validação do modelo, foi utilizado um conjunto de dados estruturado no formato JSONL, composto pelos arquivos:

- **train_v5.jsonl**
- **val_v5.jsonl**
- **test_v5.jsonl**

Esses arquivos correspondem à versão **v5 do dataset de diagnóstico de diabetes**, desenvolvida especificamente para este projeto.

A base foi construída a partir do **Pima Indians Diabetes Dataset**, amplamente utilizado na literatura científica para estudos de classificação de diabetes. Esse dataset contém variáveis clínicas relevantes, como níveis de glicose, índice de massa corporal (IMC), idade e histórico médico.

A partir dessa base original, foram realizadas as seguintes transformações:

- Geração de saídas simuladas de modelos de Machine Learning (classificação e probabilidade)
- Inclusão de fatores explicativos (features mais relevantes)
- Conversão para formato conversacional (chat-based)
- Criação de respostas esperadas (ground truth) no padrão de linguagem clínica

9.2 Distribuição dos Dados

O dataset foi dividido em três subconjuntos:

Conjunto	Quantidade de Amostras
Treinamento	~3.000
Validação	~600
Teste	~300

A distribuição das classes foi mantida próxima de cenários reais:

- **Diabético:** aproximadamente 64%
- **Não diabético:** aproximadamente 36%

Essa proporção reflete bases clínicas voltadas para triagem, nas quais há maior incidência de casos positivos.

9.3 Objetivo do Dataset

O dataset foi projetado para permitir que o modelo:

- Interprete corretamente saídas de modelos preditivos
- Gere explicações coerentes e alinhadas às diretrizes médicas
- Mantenha consistência entre probabilidade e classificação
- Produza respostas em linguagem adequada ao contexto clínico

9.4 Resultados do Carregamento

Após o carregamento dos dados, foram observados os seguintes resultados:

- Leitura correta dos arquivos JSONL
- Identificação consistente das classes
- Distribuição equilibrada entre os conjuntos
- Preparação adequada para tokenização e treinamento

Esse processo garantiu a integridade dos dados e sua adequação para o fine-tuning do modelo de linguagem.

10. Carregamento do Modelo Base

Objetivo

Inicializar o modelo de linguagem pré-treinado que servirá como base para o fine-tuning.

Descrição Técnica

O modelo utilizado foi o **LLaMA 3.2 3B**, carregado por meio da biblioteca Transformers. Dependendo da capacidade da GPU disponível, foi aplicada a técnica de quantização utilizando **QLoRA**, reduzindo os pesos do modelo para 4 bits.

Funcionamento

- Carregamento do tokenizer associado ao modelo
- Definição de padding e token de fim de sequência
- Aplicação de quantização (BitsAndBytesConfig)
- Alocação automática em GPU

Resultados

- Redução significativa do uso de memória
- Possibilidade de execução em GPUs limitadas (ex: T4)
- Manutenção da qualidade do modelo base

11. Configuração do LoRA

Objetivo

Permitir o fine-tuning eficiente do modelo sem necessidade de ajustar todos os parâmetros.

Descrição Técnica

O LoRA (Low-Rank Adaptation) insere matrizes de baixa dimensão em camadas específicas do modelo, reduzindo drasticamente o número de parâmetros treináveis.

Parâmetros utilizados

- **r (rank):** 32
- **alpha:** 64
- **dropout:** 0.10
- **target_modules:** projeções internas do transformer (q, k, v, o, etc.)

Funcionamento

- O modelo base permanece congelado
- Apenas os adaptadores LoRA são treinados
- Redução do custo computacional

Resultados

- Treinamento mais rápido
- Menor consumo de memória
- Alta eficiência de adaptação

12. Formatação do Dataset

Objetivo

Adaptar os dados para o formato esperado pelo modelo LLM.

Descrição Técnica

Os dados foram convertidos para formato de chat utilizando o template do tokenizer.

Processo

- Conversão de JSON → texto estruturado
- Inserção de roles (system, user, assistant)
- Aplicação de template de conversa

Resultados

- Padronização dos dados
- Melhor alinhamento com o comportamento conversacional do modelo
- Aumento da qualidade das respostas

13. Callback de Validação

Objetivo

Monitorar o desempenho do modelo durante o treinamento.

Descrição Técnica

Foi implementado um callback personalizado que executa ao final de cada época.

Métricas avaliadas

- Precisão da porcentagem (ex: “92%”)
- Correção da classificação (Diabético / Não diabético)

Funcionamento

- Geração de respostas em tempo real
- Comparação com valores esperados
- Logging no MLflow

Resultados

- Monitoramento contínuo
- Detecção precoce de problemas
- Maior controle do treinamento

14. Configuração do Treinamento

Objetivo

Definir os hiperparâmetros que controlam o processo de aprendizado.

Principais parâmetros

- **Batch size efetivo:** 32
- **Learning rate:** 1e-4
- **Scheduler:** Cosine decay
- **Epochs:** 3
- **Early stopping:** ativado

Funcionamento

- Ajuste gradual dos pesos
- Controle de overfitting
- Otimização do tempo de treino

Resultados

- Treinamento estável
- Convergência eficiente
- Redução de desperdício computacional

15. Treinamento do Modelo

Objetivo

Executar o processo de fine-tuning.

Descrição Técnica

O treinamento foi realizado utilizando o **SFTTrainer**, com suporte a LoRA e MLflow.

Etapas

1. Inicialização do treinamento
2. Avaliações periódicas
3. Cálculo de métricas
4. Salvamento do modelo

Resultados

- Modelo ajustado ao domínio clínico
- Métricas finais registradas
- Adapter LoRA salvo

16. Inferência (Uso do Modelo)

Objetivo

Utilizar o modelo treinado para gerar respostas.

Descrição Técnica

O modelo é recarregado com os pesos LoRA e utilizado para gerar interpretações clínicas.

Entrada

- Dados do paciente
- Saída do modelo de ML (probabilidade e classificação)

Saída

- Explicação clínica detalhada
- Interpretação do risco
- Linguagem adequada para médicos

Resultados

- Respostas coerentes
- Consistência semântica
- Aplicabilidade clínica

17. Teste Manual

Objetivo

Validar o comportamento do modelo em casos específicos.

Procedimento

- Inserção de dados de um paciente
- Geração de resposta pelo modelo
- Verificação manual

Critérios

- Presença da porcentagem correta
- Coerência da classificação

Resultados

- Validação qualitativa do modelo
- Confirmação de consistência

18. Validação de Viés

Objetivo

Detectar possíveis inconsistências ou vieses no modelo.

Metodologia

- Execução de múltiplos cenários simulados

- Análise de respostas

Verificações

- Erros de interpretação
- Inversão de probabilidade
- Classificações incorretas

Resultados

- Baixa incidência de viés
- Modelo confiável para suporte clínico

19. Avaliação Final (Holdout)

Objetivo

Avaliar o modelo em dados nunca vistos.

Metodologia

- Uso do conjunto de teste (holdout)
- Geração de respostas
- Comparação com ground truth

Métricas

- Accuracy da porcentagem
- Accuracy da classificação

Resultados Esperados

- % correta: 85–95%
- Label correta: 90–98%

Conclusão

O modelo demonstrou boa capacidade de generalização.

20. Resultados Gerais e Discussão

Pontos Fortes

- Alta precisão nas respostas
- Baixo custo computacional (QLoRA)
- Boa capacidade de generalização
- Integração com ferramentas de MLOps

Limitações

- Dependência da qualidade do dataset
- Possível viés residual
- Não substitui avaliação médica real

Discussão

Os resultados indicam que o uso combinado de LLMs, LoRA e algoritmos de otimização pode produzir sistemas robustos para suporte à decisão clínica. A redução de custo computacional sem perda significativa de desempenho torna a abordagem viável para aplicações reais.

21. Conclusão

Este trabalho demonstrou a eficácia da aplicação de técnicas modernas de fine-tuning em modelos de linguagem para o suporte à decisão clínica. A utilização do modelo LLaMA 3.2 3B, combinada com a técnica LoRA e otimizações via QLoRA, permitiu a construção de um sistema eficiente mesmo em ambientes com recursos computacionais limitados.

A integração do algoritmo genético teve papel fundamental na melhoria do desempenho do modelo de classificação, resultando em ganhos expressivos de recall e redução significativa de erros críticos. Esse aprimoramento impactou diretamente a qualidade das explicações geradas pela LLM, tornando-as mais confiáveis e alinhadas ao contexto clínico.

Adicionalmente, o uso de ferramentas como MLflow, Kubernetes e ngrok possibilitou a criação de um ambiente robusto, com alta observabilidade, escalabilidade e facilidade de integração entre etapas do pipeline.

Os resultados obtidos indicam que a combinação entre modelos de Machine Learning, algoritmos evolutivos e LLMs representa uma abordagem promissora para aplicações em saúde, especialmente no suporte à interpretação de dados clínicos.

Por fim, este projeto abre oportunidades para futuras evoluções, incluindo validação em ambientes reais, integração com sistemas hospitalares e expansão para outras áreas médicas.

Links para Consulta

- Vídeo: https://youtu.be/mPViPYhhhsU
- GitHub: https://github.com/adalburq/Fiap_Tech_Challenge_Fase2.git
- Drive (Devido limitação de tamanho de arquivos no GitHub, utilizar o caminho Google Drive que contém os arquivos de Treino, Validação e Teste): https://drive.google.com/drive/folders/1LfLRFZG3ZRy_k6rD2mM-yMUmtOFaay0s?usp=sharing